

APUNTES TEMA 5: COLECCIONES

COMPRENDER CONVENCIONES

Letra	Convención habitual	Significado
T	Type	El tipo principal de entrada
R	Return	El tipo de retorno/resultado
S	Second	Un segundo tipo (de entrada, generalmente)
U	Tercer tipo	Cuando se necesita un tercero
E	Element	Elemento (usado en colecciones)
K	Key89	Clave (usado en mapas)
V	Value	Valor (usado en mapas)

API COLLECTIONS

COLLECTION <E> ⇒ INTERFAZ LIST

⇒ Esta interfaz nos permite funcionalidades como:

- Acceso posicional
- Búsqueda
- Iteración extendida
- Operaciones sobre un rango de elementos

⇒ En este caso la letra <E> hará referencia a **elementos**.

Para importar la colecciónS:

```
import java.util.List;
```

QUE ENCONTRAMOS EN LIST:

1. ArrayList <E>
2. LinkedList <E>

ENTENDER ARRAYLIST

⇒ Estos hacen falta importar su librería:

```
import java.util.ArrayList;
```

Ejemplo de sintaxis:

List <Elemento (básicamente su tipo: producto, Integer...)> (nombre de la lista) = new ArrayList <Elemento> ();

```
List <Integer> listilla = new ArrayList<Integer>();
```

A la hora de instanciar listas en clases encontraremos dos versiones:

```
* private List<Calificacion> listaNotas;  
* Si lo hacemos de este metodo, esta lista de notas no estara instanciada y puede  
* derivar en problemas, para solucionarlo  
* hay que crear un constructor con la lista.  
  
* private List <Calificacion> ListAsigntaturas = new ArrayList <Calificacion>();  
* Aqui lo que estamos es instanciado directamente aqui
```

Métodos que hallamos en ArrayList

1. Agregar: .add()

2. Limpiar lista: `.clear()`
3. Eliminar object: `.remove(int index)`
4. Setear objeto: `.set(int index, E element)`
5. Coger object: `.get(int index)`

[más métodos](#)

Métodos para ordenar Listas

Interfaz a utilizar ⇒ [comparable](#)

1. `compareTo ((Tipo de objeto) (instancia))`

COLLECTION <E> ⇒ INTERFAZ SET

⇒ Son una especie de lista en la que **no disponemos de índice**, básicamente podemos ponerlo en analogía como si fuera una bolsa de basura

Características a recordada:

- Los sets no admiten duplicados
- No hay orden de inserción

¿Cómo se inicializan los sets? ⇒ Siempre lo definiremos con la primera parte con (**Set**).

```
Set<Alumnos> Listado = new HashSet<>();
```

⇒ Si lo hacemos de esta forma no hace falta inicializarlo en el main, ya que directamente el constructor lo hace.

Formas de ordenar en los sets ⇒ esto lo haremos mediante la formas naturales que le asignamos en la clase poyo.

Esto lo hacemos con la interfaz Comparable

```
public class Alumnos implements Comparable<Alumnos> {
```

Esta clase se implementa un método con el fin de comparar:

```
@Override
public int compareTo(Alumnos o) {
    // TODO Auto-generated method stub
    return 0;
}
```

Comprender ⇒ Este método importado de la interfaz tiene como fin establecer un orden de orden de forma natural, para comparar podemos modificar el método, ejemplo:

```
@Override
public int compareTo(Alumnos alumno) {
    return this.nombre.compareTo(alumno.getNombre());
}
```

Comprender ⇒ Si la forma es natural (forma de ordenar una colección) es básicamente cuando no se le dice ningún criterio, realmente el que ordena es el método **sort** (de collection) busca en la clase (Poyo) que haya un método de criterio. Si no decimos nada es que esa es su forma natural de ordenar.

Significa ⇒ Si desde la principal llamamos al `collections.sort(clase poyo)`

Usamos el `sort` que ordena, y el método que compara. Si cuando lo llamamos solo usa la colección, como no le damos nada más se va a la clase que define ese objeto. Ahí buscará que tenga implementada OBLIGATORIAMENTE la interfaz `Comparable` que tendrá que tener el método de la interfaz `Comparable` el **método reescrito (`compareTo`)**, a esta se le pasa un objeto de esa misma clase, devolverá un `int` (normalmente 0, 1, -1); el programador define la forma en la que se ordena (tal vez dorsal de mayor a menor, de menor a mayor, pares... según nos lo pidan).

Que ocurre en el caso de que queramos tener otras maneras de ordenar??

⇒ Lo primero es comprender que solo hay **un orden natural**, y en ella hay que hacer una clase (public class) por cada forma de ordenar no natural.

Importante ⇒ Para los **orden no naturales** hay que implementar la interfaz **comparator**

```
public class CompararPorMarca implements Comparator<Telefono>{
```

⇒ A diferencia del `Comparable` al método de esta clase deberemos de pasar **dos objetos** a diferencia de con la *natural* que era solo **un objeto**.

Método autogenerado de la interfaz

```
public int compare(Telefono o1, Telefono o2) {
    // TODO Auto-generated method stub
    return 0;
}
```

⇒ Dentro de este método (*para no naturales*) implementamos la lógica de ordenación.

Lógica de método (ejemplo)

```
public int compare(Corredor o1, Corredor o2) {  
    // TODO Auto-generated method stub  
  
    double marcac1 = o1.getMarca();  
    double marcac2 = o2.getMarca();  
  
    if (marcac2 > marcac1) {  
        return -1;  
    } else {  
        if (marcac1 > marcac2) {  
            return 1;  
        }  
        return 0;  
    }  
}
```

Para aplicar este método en el main debemos comprender que hay diferentes formas de indicar la forma de ordenación.

Para naturales: si analizamos solo le pasamos la lista de objetos.

```
Collections.sort(corredores );
```

Para no naturales: si analizamos le pasamos tanto la lista como la instancia de la clase

```
Collections.sort(corredores, new compararMarca());
```

En el caso de los sets (los cuales no están hechos para ser ordenados debido a su naturaleza). En estos casos habría que transformar ese **sets** en una lista y a partir de ahí utilizarlo.

Comprender ⇒ Aunque esto esté en el sets no se suelen utilizar esto en los sets. Directamente saltara un error.

Método para sacar todos los items del set:

```
public Set<Alumnos> findAll () {  
    return Collections.unmodifiableSet(Listado);  
}
```

⇒ Este método nos permitirá ahorrarnos crear un bucle.

(MIRAR LOS ITERATORS EN GENERAL (ENTRAN EN EL EXAMEN))

COLLECTION $\langle K, V \rangle \Rightarrow$ INTERFAZ MAP